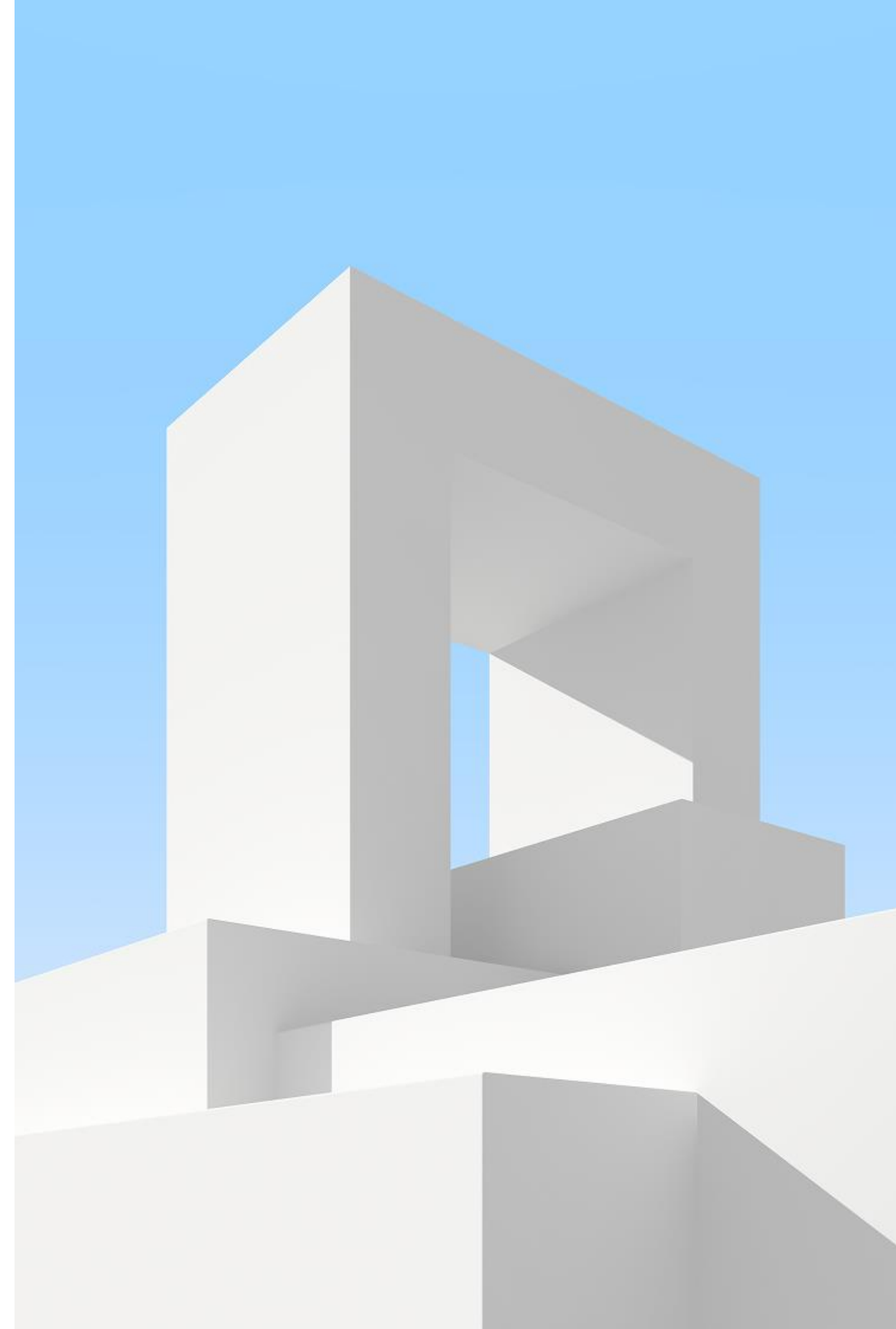


Clean Code, SOLID Prensipleri ve Katmanlı Mimari ile Sürdürülebilir Mobil

Clean Code, SOLID Prensipleri ve Katmanlı Mimari

Ahmet Bekir Bakkal



```

func animateTo(_ aTo: NSNumber, _ aFrom: NSNumber? = nil, _ shouldShowPercentage: Bool = false) {
    if self.text == nil {
        if aFrom == nil {
            self.text = "0"
        } else {
            self.text = "\(aFrom!)"
        }
    }
    from = aFrom != nil ? aFrom! : NSNumber(value: Float(self.text!) ?? 0 as Float)
    to = aTo
    showPercentage = shouldShowPercentage
    let link: CADisplayLink = CADisplayLink(target: self, selector: #selector(animateNumber))
    startTime = CACurrentMediaTime() as AnyObject?
    link.add(to: RunLoop.current, forMode: RunLoop.Mode.common)
}

@objc func animateNumber(_ link: CADisplayLink) {
    let duration: Double = 1.0
    let durationTime: Float = Float((link.timestamp - (startTime as? TimeInterval)!) / duration)
    if durationTime >= 1.0 {
        let text: String = "\(Int(to as? Int)!) " + (showPercentage! ? "%" : "")
        self.text = text
        link.remove(from: RunLoop.current, forMode: RunLoop.Mode.common)
    } else {
        let fromFloat = from as! Float
        let toAsFloat = to as? Float ?? 0
        let current = toAsFloat < fromFloat ? fromFloat - (100 - toAsFloat) * durationTime : fromFloat + (toAsFloat - fromFloat) * durationTime
        self.text = "\(Int(current))"
    }
}

```

Her build'de 16 saniye

```

func animateTo(_ aTo: NSNumber, _ aFrom: NSNumber? = nil, _ shouldShowPercentage: Bool = false) {
    if self.text == nil {
        if aFrom == nil {
            self.text = "0"
        } else {
            self.text = "\(aFrom!)"
        }
    }
    from = aFrom != nil ? aFrom! : NSNumber(value: Float(self.text!) ?? 0 as Float)
    to = aTo
    showPercentage = shouldShowPercentage
    let link: CADisplayLink = CADisplayLink(target: self, selector: #selector(animateNumber))
    startTime = CACurrentMediaTime() as AnyObject?
    link.add(to: RunLoop.current, forMode: RunLoop.Mode.common)
}

@objc func animateNumber(_ link: CADisplayLink) {
    let duration: Double = 1.0
    let durationTime: Float = Float((link.timestamp - (startTime as? TimeInterval)!) / duration)
    if durationTime >= 1.0 {
        let text: String = "\(Int(to as? Int)!) " + (showPercentage! ? "%" : "")
        self.text = text
        link.remove(from: RunLoop.current, forMode: RunLoop.Mode.common)
    } else {
        let fromFloat = from as! Float
        let toAsFloat = to as? Float ?? 0
        let current = toAsFloat < fromFloat ? fromFloat - (100 - toAsFloat) * durationTime : fromFloat + (toAsFloat - fromFloat) * durationTime
        self.text = "\(Int(current))"
    }
}

```

Her build'de 4 saniye

```

func animateTo(_ aTo: NSNumber, _ aFrom: NSNumber? = nil, _ shouldShowPercentage: Bool = false) {
    if self.text == nil {
        if aFrom == nil {
            self.text = "0"
        } else {
            self.text = "\(aFrom!)"
        }
    }
    from = aFrom != nil ? aFrom! : NSNumber(value: Float(self.text!) ?? 0 as Float)
    to = aTo
    showPercentage = shouldShowPercentage
    let link: CADisplayLink = CADisplayLink(target: self, selector: #selector(animateNumber))
    startTime = CACurrentMediaTime() as AnyObject?
    link.add(to: RunLoop.current, forMode: RunLoop.Mode.common)
}

@objc func animateNumber(_ link: CADisplayLink) {
    let duration: Double = 1.0
    let durationTime: Float = Float((link.timestamp - (startTime ?? 0) * 1000) / 1000) / duration
    if durationTime >= 1.0 {
        let text: String = "\(Int(to as? Int)!) " + (showPercentage ? "%": "")
        self.text = text
        link.remove(from: RunLoop.current, forMode: RunLoop.Mode.common)
    } else {
        let fromFloat = from as! Float
        let toAsFloat = to as? Float ?? 0
        let current = toAsFloat < fromFloat ? fromFloat - (100 - toAsFloat) * durationTime : fromFloat + (toAsFloat - fromFloat) * durationTime
        self.text = "\(Int(current))"
    }
}

```

İçindekiler

Eğitimin Vizyonu

Yazılımın ilk gününde değil, gelecekte de sürdürülebilir olmasının yollarını keşfedeceğiz.

SQA & SDLC & OOP

Yazılım kalite özelliklerini, yazılım geliştirme yaşam döngüsünü ve nesne yönelimli programlamanın temellerini ele alacağız.

SOLID Prensipleri & Cohesion ve Coupling

Değişime dayanıklı kod yapısı ve katmanlı mimari prensiplerini öğreneceğiz.

Gitflow & Sonuç

Modern geliştirme süreçleri ve sürdürülebilir kod yönetimi stratejilerini inceleyeceğiz.

Eğitimin Amacı ve Vizyonu

Ana Hedefimiz

Yazılımın ilk gününde değil, gelecekte de hâlâ okunabilir, test edilebilir ve büyümeye elverişli olmasını sağlamak. Sadece kod yazmak değil, kod yaşatmak.



Clean Code

Sade ve anlaşılır yapı ile okunabilirliği maksimize eden kod yazma teknikleri ve en iyi pratikler.



SOLID & Mimari

Değişime dayanıklı yapı ve sürdürülebilir büyüme sağlayan geleceğe uyumlu uygulama mimarisi.



Ahmet Bekir BAKKAL

iOS Developer

Marmara Üniversitesi

Bilgisayar Mühendisliği (2010-2015)

Kuveyt Türk (2015-2017)

Ziraat Teknoloji (2017-2021)

Turkcell (2021-Halen)



Yazılım Kalitesi Nedir?

Yazılım kalitesi, bir sistemin kullanıcı, müşteri ve geliştirici beklentilerini ne kadar karşıladığını belirleyen ölçütler bütünüdür. ISO 9001.



Yalnızca çalışan bir yazılım değil, sürdürülebilir, güvenli ve ölçeklenebilir bir yazılım üretmek.



Yazılım kalitesi, sadece doğru kod yazmak değil; geleceğe dayanıklı bir sistem inşa etmektir.



Bu ölçütler Software Quality Attributes olarak adlandırılır ve sistemin başarısını belirler.

Temel Kalite Özellikleri

Yazılım kalitesini belirleyen temel özellikler ve açıklamaları. Bir bankacılık uygulamasının güvenli, hızlı ve kolay kullanılabilir olması bu niteliklerin birleşimidir.

Özellik	Kısa Açıklama
Maintainability	Kodun kolayca okunması, düzenlenmesi ve genişletilmesi
Scalability	Kullanıcı veya yük artışına rağmen performansın korunması
Reliability	Uzun süre kararlı ve hatasız çalışabilmesi
Performance	Hızlı tepki süreleri ve verimli kaynak kullanımı
Security	Veri ve kullanıcı güvenliğini sağlama becerisi
Usability	Kolay öğrenilebilir ve kullanıcı dostu arayüz
Testability	Kodun kolayca test edilebilmesi
Portability	Farklı platformlarda çalışabilme yeteneği

Yazılım Bir Ağaç Gibidir



Kökler: Temiz Kod

Kökler sağlam değilse ağaç büyüyemez. Temiz kod, yazılımın temelini oluşturur. Okunabilir, anlaşılır ve sürdürülebilir kod yapısı tüm sistemin sağlam durmasını sağlar.

Gövde: SOLID Prensipleri

SOLID prensipleri yazılımın ana gövdesini oluşturur. Değişime dayanıklı, genişletilebilir ve bakım yapılabilir kod mimarisi için vazgeçilmez yapı taşlarıdır.

Yazılım Geliştirme Yaşam Döngüsü (SDLC)

Mobil uygulama geliştirme sürecinde kalite, testte değil tasarımda başlar.
Her aşamanın kendine özgü sorumlulukları ve çıktıları vardır.

Aşama	Açıklama	Mobil Örnek	Çıktı
Requirement	Kullanıcı ihtiyaçlarının belirlenmesi	QR kodla bilet gösterme	Gereksinim dokümanı
Design	Sistem ve UI/UX tasarımı	Ekran akışı ve API planı	Tasarım prototipleri
Development	Kodlama ve implementasyon	UI, ViewModel, Network kodları	Çalışan uygulama
Testing	Kalite güvence testleri	Unit ve UI testleri	Test raporları
Deployment	Canlı ortama yayınlama	App Store yayını	Yayınlanmış uygulama

"

**Kalite testte deęil, kodun i tasarımında başlar.
Bir mimari düşünölmeden yazılan kod, ileride teknik
bor olarak geri döner.**

Bu temel gerek, sürdürülebilir yazılım geliřtirmenin en önemli prensiplerinden birini ortaya koyar.

Baştan doęru tasarım yapmak, sonradan düzeltmekten her zaman daha etkili ve maliyetlidir.

Nesne Yönelimli Programlama

Encapsulation (Kapsülleme)

Veriyi korumak ve dış dünyadan soyutlamak için kullanılan temel prensip ve uygulamaları

Inheritance (Kalıtım)

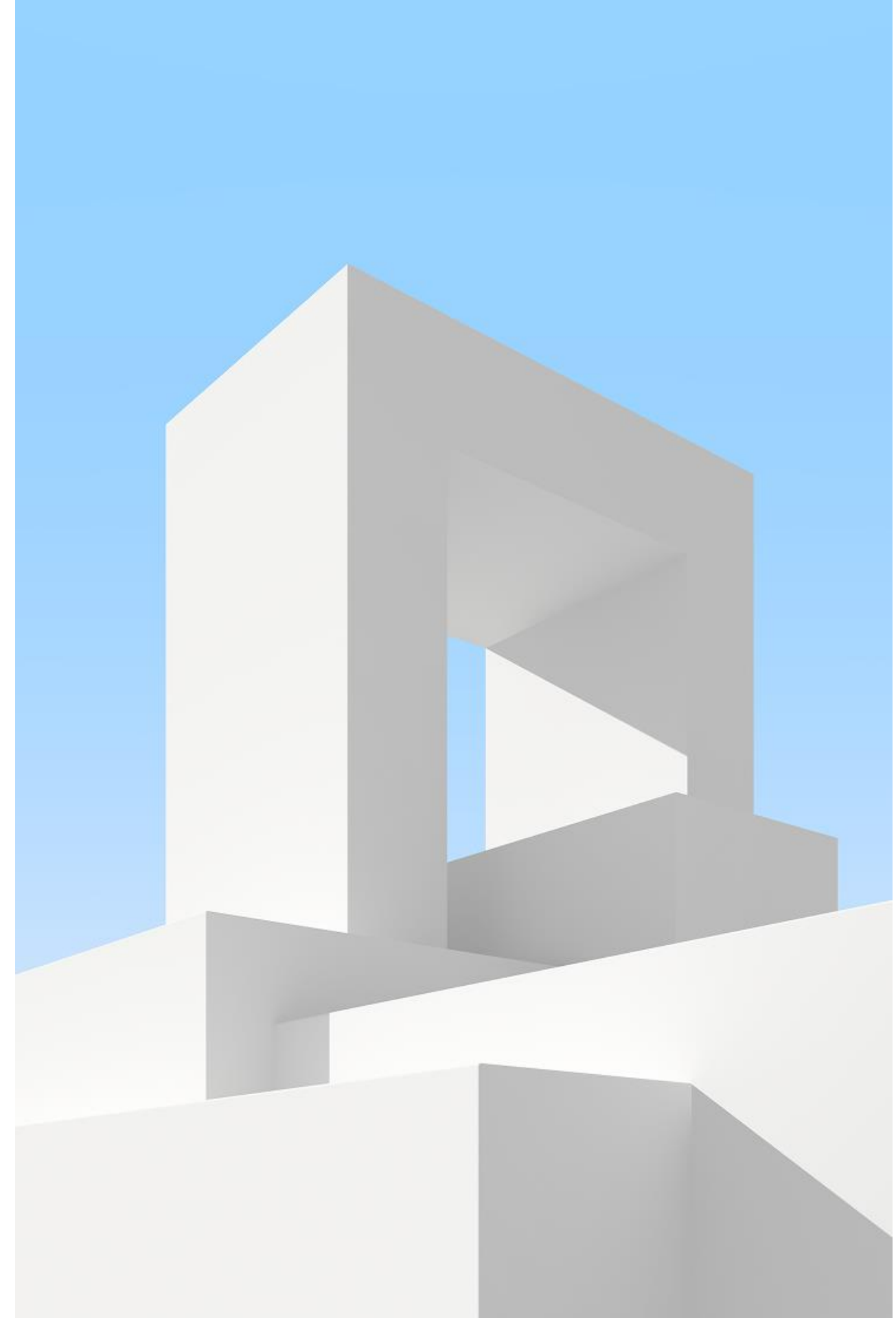
Kodun tekrarını azaltmak ve ortak davranışları paylaşmak için kullanılan kalıtım mekanizması

Polymorphism (Çok Biçimlilik)

Aynı işlemin farklı nesneler üzerinde farklı şekillerde çalışabilmesini sağlayan yaklaşım

Abstraction (Soyutlama)

Karmaşık sistemleri sadeleştirip sadece gerekli detayları gösteren soyutlama prensipleri



Encapsulation (Kapsülleme)

Temel Kavram

Nesnenin verileri yalnızca o nesne tarafından kontrol edilir. Dışarıdan erişim getter/setter gibi kontrollü yollarla sağlanır. Bu yaklaşım kodun güvenliğini artırır.



Güvenlik

Verileri dış müdahalelerden koruyarak sistem güvenliğini artırır ve istenmeyen değişiklikleri önler



Bakım Kolaylığı

Kod değişikliklerinin etkisini sınırlayarak sürdürülebilirlik ve bakım kolaylığını sağlar



Inheritance (Kalıtım)

Kalıtım Prensibi

Bir sınıf başka bir sınıfın özelliklerini ve davranışlarını devralır. Kodun tekrarını azaltarak yeniden kullanılabilirliği artırır.



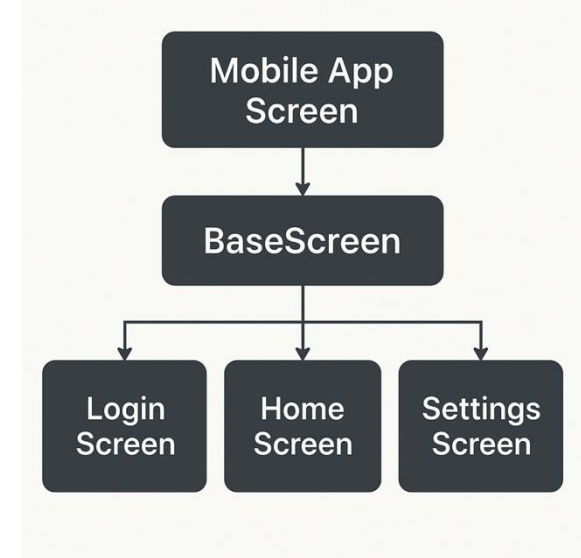
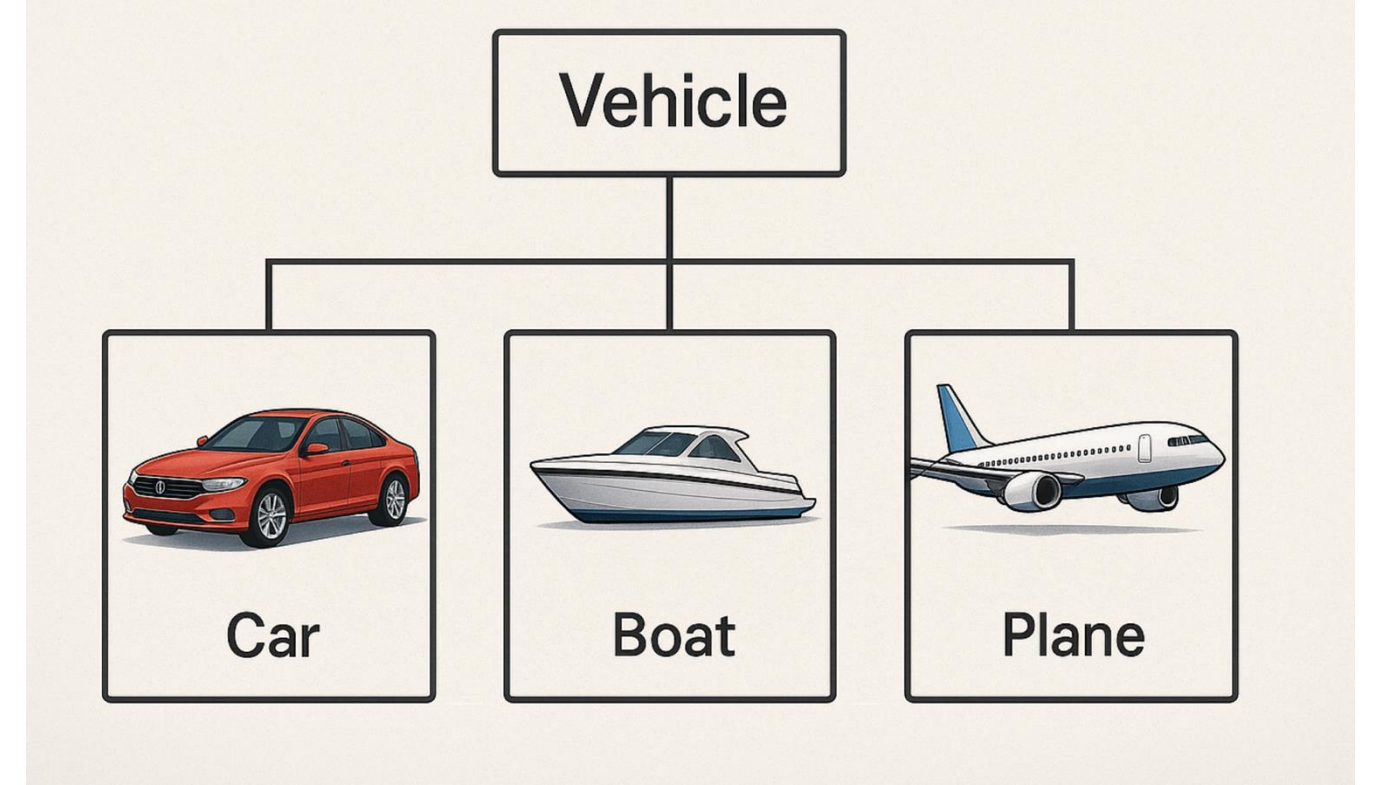
Kod Tekrarını Azaltma

Ortak özellikleri bir kez tanımlayarak tüm alt sınıflarda kullanma imkanı sağlar



Genişletilebilirlik

Mevcut koda dokunmadan yeni özellikler ekleyebilme ve sistemin geliştirilmesi



Polymorphism (Çok Biçimlilik)

Polimorfizm Kavramı

Aynı isimli bir metot, farklı sınıflarda farklı biçimlerde davranabilir. Bu esnek ve modüler mimari sağlar.



Esneklik

Aynı arayüz üzerinden farklı davranışlar sergileyebilen nesneler oluşturma imkanı



Modülerlik

Değişen gereksinimlere kolayca uyum sağlayabilen genişletilebilir sistem tasarımı



Abstraction (Soyutlama)

Soyutlama Prensipleri

Karmaşık sistemleri sadeleştirip sadece gerekli detayları gösterir. Nasıl yapıldığını değil, ne yaptığını biliriz.



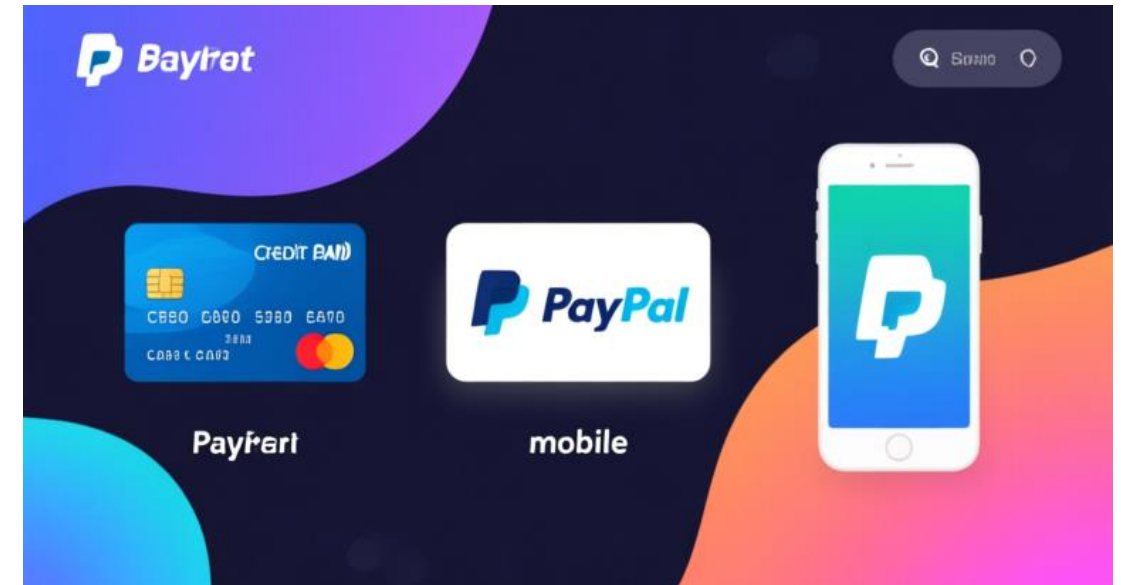
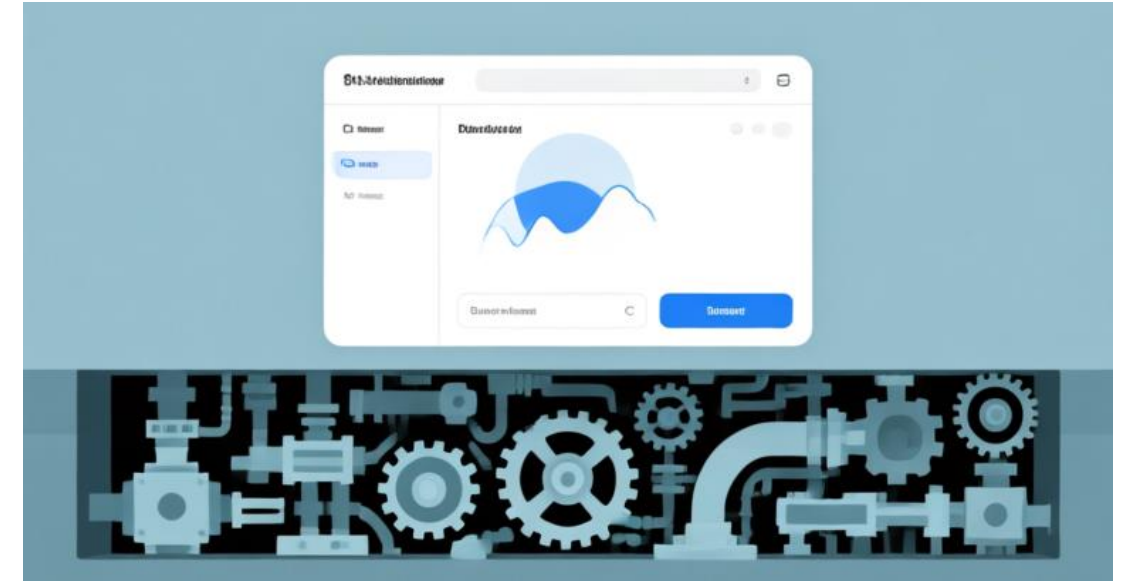
Karmaşıklığı Gizleme

İç detayları gizleyerek kullanıcıya sadece gerekli arayüzü sunar



Bağımsızlık

Kodun okunabilirliğini artırır ve sistem bileşenlerinin bağımsızlığını sağlar



"

OOP: Modüler, Sürdürülebilir ve Geniřletilebilir Kodun Temelidir

Bu 4 prensip, yazılımın ölçeklenebilir, bakımı kolay ve hatalara karşı dayanıklı olmasını sağlar.

Clean Code, SOLID ve Katmanlı Mimari gibi yaklaşımlar bu temel üzerine inşa edilir.

Things Smell...

- University • Dorm • Manager • Desk • Chat • Slack • Git • Jenkins • Pizza Box
- Air Conditioner • Colleague • Whiteboard • Idea • Bug • Code • Architecture
- Requirements • Process • Test • Comments • Functions • Naming • Team • PR
- Communication • Documentation • Deadlines • Merge • Review • Design • Office
- Meeting • Coffee • Laptop • Keyboard

Code Smell Nedir?

Code Smells Are Symptoms, Not Bugs

Code smell bir hata değildir; potansiyel bir tasarım veya bakım problemi göstergesidir. Kod çalışır ama anlaşılması, test edilmesi ve geliştirilmesi zorlaşır.



Duplicated Code

Aynı mantığın birçok yerde tekrar edilmesi. DRY prensibini ihlal eder ve bakım zorluklarına yol açar.



Large Class

Her şeyi yapan God Class. Single Responsibility Principle'i ihlal eder ve çok fazla sorumluluk alır.



Long Method

100 satırlık dev fonksiyonlar. Anlaşılması zor, test edilmesi karmaşık ve değiştirilmesi riskli kod blokları oluşturur.

Yaygın Code Smell Türleri

Code Smell Belirti Kategorileri

Martin Fowler'ın kategorilendirmesine göre code smell'ler yazılım kalitesini olumsuz etkileyen ve teknik borcu artıran tasarım kararlarının belirtileridir.

Feature Envy

- Bir method başka sınıfla aşırı ilgileniyorsa
- Kendi verilerinden çok başkasının verisini kullanıyor
- Encapsulation prensibini ihlal ediyor

Primitive Obsession

- Basit veri tipleriyle karmaşık yapılar kurmaya çalışmak
- String, int gibi primitiflerle domain modelleme
- Type safety eksikliği ve anlam kaybı

Comments Overload

Kod kötü olduğu için yorum satırları ile açıklama ihtiyacı

Magic Numbers

Anlamı net olmayan sayıların kodda dolaşması

Switch Statements

Polymorphism yerine açık switch-case kullanımı

Code Smell Tespiti ve Çözüm Süreci

Erken Tespit

Code review, static analysis araçları ve complexity metrikleri ile code smell'leri erkenden fark edin.



Refactoring Planlama

Clean Code, SOLID prensipleri ve refactoring teknikleri ile sistemli iyileştirme planı oluşturun.

Sürekli İyileştirme

Kod kalitesini korumak için düzenli refactoring ve code review süreçlerini benimseyin.



Teknik Borç Önleme

Coding standard'ları, automated testing ve CI/CD pipeline ile teknik borcun birikmesini engelleyin.

"

Kod kokusu, bugün çalışan ama yarın sorun çıkaracak kod yapısıdır. Bulaşık makinesinde unutulmuş tabak gibidir.

Kod kokularını erken tespit etmek ve refactor etmek, yazılımın sağlığını korumak için kritik öneme sahiptir.

”

**Bugün zamandan kazanmak için yaptığın her kısayol,
yarın sana gider olarak geri döner.**

— Technical Debt Gerçeği

Technical Debt, yazılım geliştirmede en büyük tuzaklardan biridir. 'Bunu şimdilik böyle yapalım' yaklaşımı, zamanla sistemin sürdürülebilirliğini ciddi şekilde tehdit eder ve geliştirme maliyetlerini katlar.

Cohesion & Coupling: Mükemmel Denge

Yüksek Cohesion Hedefi

Yüksek Cohesion

Modülün kendi içinde tutarlılığı maksimum seviyede olmalıdır.

- Her sınıf kendi görevine odaklanır.
- İlgili fonksiyonlar bir arada bulunur.
- Tek bir amaç için tasarlanmıştır.

Düşük Coupling Hedefi

Düşük Coupling

Modüller arası bağımlılık minimum seviyede tutulmalıdır.

- Değişiklik diğer modülleri etkilemez.
- Bağımsız test edilebilir yapıdadır.
- Kolay değiştirilebilir ve genişletilebilir.

SOLID Prensipleri

01. Single Responsibility

Bir sınıfın sadece tek bir sorumluluğu olmalı ve değişme nedeni yalnızca bir olmalıdır.

02. Open/Closed Principle

Kod yeni davranışlar eklemeye açık, mevcut davranışları değiştirmeye kapalı olmalıdır.

03. Liskov Substitution

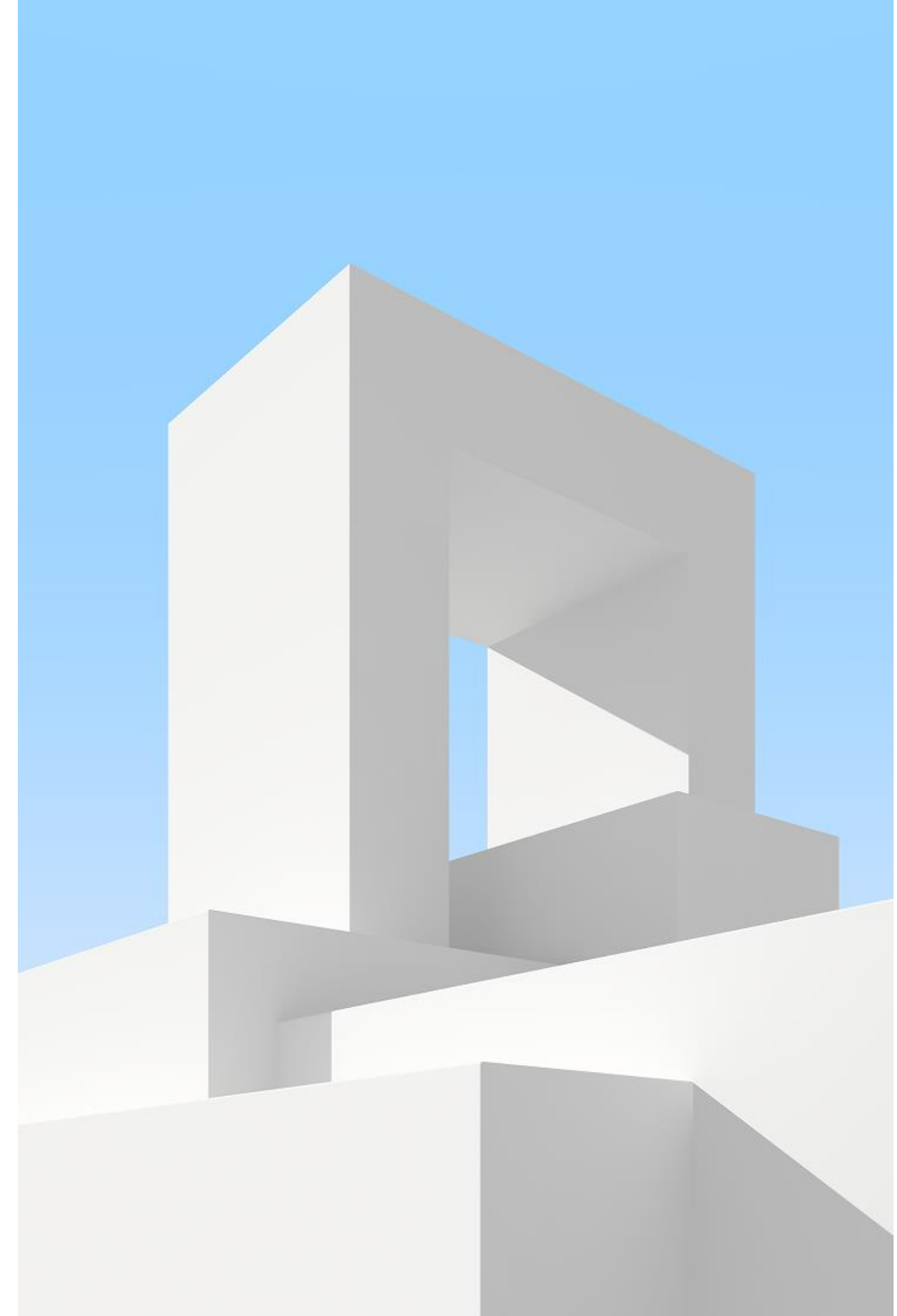
Alt sınıflar, üst sınıfın yerine geçebilmeli ve programın doğruluğu bozulmamalıdır.

04. Interface Segregation

Sınıflar kullanmadıkları metotlara bağımlı olmamalı, odaklı arayüzler kullanılmalıdır.

05. Dependency Inversion

Sınıflar kullanmadıkları metotlara bağımlı olmamalı, odaklı arayüzler kullanılmalıdır.



Single Responsibility Principle (SRP)



SRP Uygulaması

- Bir sınıfın sadece **tek bir sorumluluğu** olmalı.
- Bir sınıfın değişme nedeni **yalnızca bir** olmalıdır.
- Sınıflar “her işi yapan” hale geldiğinde kodun yönetimi zorlaşır. Bir modül hem rapor üretip hem kaydediyor hem de e-posta atıyorsa, herhangi bir değişiklik (örneğin e-posta gönderme mantığındaki güncelleme) tüm sınıfı etkiler. Bu durum *bağımlılık zinciri* ve *yan etkiler* oluşturur.

SRP ihlali

- UserManager hem giriş, hem kayıt, hem email gönderiyor
- Tek sınıfta çoklu sorumluluk: login(), register(), sendEmail()
- Bir değişiklik tüm sistemi etkileyebilir



Single Responsible Principle – Kod Örneği

```
class ReportManager {  
    func generateReport() { /* rapor oluşturur */ }  
    func saveToDatabase() { /* veritabanına kaydeder */ }  
    func sendEmail() { /* mail gönderir */ }  
}
```

```
class ReportGenerator {  
    func generate() { /* rapor oluşturur */ }  
}  
  
class ReportSaver {  
    func save(_ report: Report) { /* veritabanına kaydeder */ }  
}  
  
class ReportMailer {  
    func send(_ report: Report) { /* mail gönderir */ }  
}
```

Open/Closed Principle (OCP)



Kod yeni davranışlar eklemeye açık, ama mevcut davranışları değiştirmeye kapalı olmalıdır.

Bu prensip, kodun değişiklikten korkmadan büyüyebilmesini sağlar. Yeni bir özellik eklemek için eski sınıfları değiştirmek yerine, mevcut yapıyı genişleterek ilerlemeliyiz.

Bu da genellikle polimorfizm ve soyutlama (protocol/interface) ile yapılır.

OCP ihlali

- Her yeni kod için mevcut kodun değiştirilmesi
- if-else zinciri sürekli büyümesi
- Yeni özellik eklemek mevcut sistemin riske atılması ve büyük commit'lere sebep olması.



Open/Closed Principle – Kod Örneği

```
class PaymentProcessor {  
    func process(paymentType: String) {  
        if paymentType == "CreditCard" { /* ... */ }  
        else if paymentType == "PayPal" { /* ... */ }  
        else if paymentType == "Crypto" { /* ... */ }  
    }  
}
```

```
protocol Payment {  
    func pay(amount: Double)  
}  
  
class CreditCardPayment: Payment {  
    func pay(amount: Double) { print("Paid with Credit Card") }  
}  
  
class PayPalPayment: Payment {  
    func pay(amount: Double) { print("Paid with PayPal") }  
}  
  
class PaymentProcessor {  
    func process(_ payment: Payment, amount: Double) {  
        payment.pay(amount: amount)  
    }  
}
```

Liskov Substitution Principle (LSP)



Bir alt sınıf, üst sınıfın yerine geçebilmeli ve programın doğruluğu bozulmamalıdır.

Alt sınıflar, ebeveynlerinin söz verdiği davranışları bozarsa, kalıtım bir yük haline gelir.

Bu ilke “doğru miras” kavramını anlatır.

“Eğer bir alt sınıf, üst sınıfın sözünü tutmuyorsa, o kod kokmaya başlamıştır.”

LSP – Kod Örneği

```
class Bird {  
    func fly() { print("Flying") }  
}  
  
class Penguin: Bird {  
    override func fly() { fatalError("Penguins can't fly!") }  
}
```

```
protocol Bird { }  
  
protocol FlyingBird: Bird {  
    func fly()  
}  
  
class Eagle: FlyingBird {  
    func fly() { print("Eagle flying") }  
}  
  
class Penguin: Bird {  
    func swim() { print("Penguin swimming") }  
}
```

Interface Segregation Principle

Temel Sorun

Zorunlu ancak kullanılmayan metotlar gereksiz bağımlılıklar yaratır ve sistem karmaşıklığını artırarak kodun sürdürülebilirliğini olumsuz etkiler.

Bağımlılık Sorunu

- Kullanılmayan metotlara zorunlu bağımlılık
- Küçük değişiklikler büyük etkilere neden olur
- Tüm tüketiciler yeniden derleme/test gerektirir

Test ve Geliştirme

- Mock/Stub nesneleri gereksiz yere şişer
- Testler kırılgan hale gelir
- God Interface anti-pattern ortaya çıkar

Çözüm

Küçük, amaç odaklı arayüzler tasarlayın.

Hedef

Düşük coupling, yüksek cohesion sağlayın.

Sonuç

Esnek ve sürdürülebilir sistem mimarisi.

Interface Segregation – Kod Örneği

```
protocol Worker {  
    func work()  
    func eat()  
}  
  
class Robot: Worker {  
    func work() { print("Working") }  
    func eat() { fatalError("Robots don't eat!") }  
}
```

```
protocol Workable {  
    func work()  
}  
  
protocol Eatable {  
    func eat()  
}  
  
class Human: Workable, Eatable {  
    func work() { print("Working") }  
    func eat() { print("Eating") }  
}  
  
class Robot: Workable {  
    func work() { print("Working efficiently") }  
}
```

Interface Segregation – Kod Örneği 2

```
protocol MediaDownloader {
    func downloadImage(url: URL)
    func downloadVideo(url: URL)
    func downloadPDF(url: URL)
}

final class FeedCellViewModel {
    private let downloader: MediaDownloader
    init(downloader: MediaDownloader) {
        self.downloader = downloader
    }

    func load() {
        // Sadece görsel indiriyor ama video/PDF metotlarına da bağımlı
        downloader.downloadImage(url: URL(string: "https://img")!)
    }
}
```

```
protocol ImageDownloadable { func downloadImage(url: URL) }
protocol VideoDownloadable { func downloadVideo(url: URL) }
protocol PdfDownloadable { func downloadPDF(url: URL) }

final class ImageDownloader: ImageDownloadable {
    func downloadImage(url: URL) { /* ... */ }
}

final class FeedCellViewModel {
    private let imageDownloader: ImageDownloadable
    init(imageDownloader: ImageDownloadable) {
        self.imageDownloader = imageDownloader
    }

    func load() {
        imageDownloader.downloadImage(url: URL(string: "https://img")!)
    }
}
```

Dependency Inversion Principle (DIP)

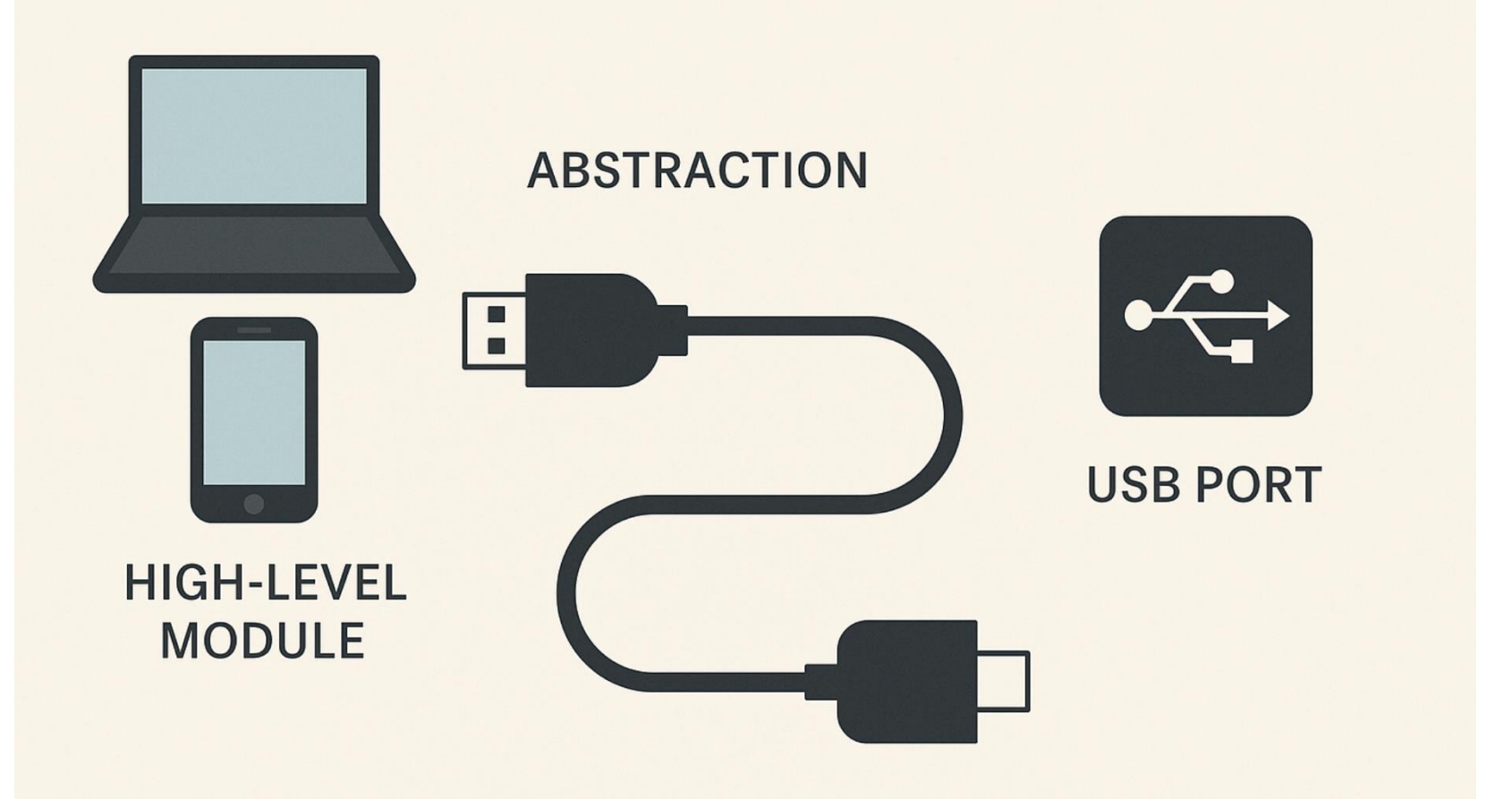
Üst seviye modüller (iş mantığı, business logic), alt seviye detaylara (veritabanı, dosya sistemi, network) doğrudan bağımlı olmamalıdır.

Her iki taraf da *soyutlamalara* (abstraction, protokol, interface) bağımlı olmalıdır.

- Bağımlılıklar yukarıdan aşağıya değil, soyutlamalar üzerinden tersine çevrilmelidir.
- Bu sayede üst katmanlar belirli bir teknolojiye *bağlanmaz*.
- Uygulama kolay test edilir, teknoloji bağımlılıklarından kurtulur.

💡 Özet:

“Kodunuz veritabanına değil, onun yeteneklerine bağlı olsun.”



Dependency Inversion Principle – Kod Örneği

```
class MySQLDatabase {  
    func save(data: String) {  
        print("Saving to MySQL")  
    }  
}  
  
class UserRepository {  
    private let db = MySQLDatabase()  
  
    func saveUser(_ user: String) {  
        db.save(data: user)  
    }  
}
```

```
protocol Database {  
    func save(data: String)  
}  
  
class MySQLDatabase: Database {  
    func save(data: String) {  
        print("Saving to MySQL")  
    }  
}  
  
class MongoDBDatabase: Database {  
    func save(data: String) {  
        print("Saving to MongoDB")  
    }  
}  
  
class UserRepository {  
    private let db: Database  
  
    init(db: Database) {  
        self.db = db  
    }  
  
    func saveUser(_ user: String) {  
        db.save(data: user)  
    }  
}
```

SOLID Prensiplerinin Bütünlüğü

SOLID prensipleri birbirleriyle uyumlu çalışarak yazılımın kalitesini artırır. Her prensip farklı bir odak noktasına sahiptir ancak hepsi birlikte temiz, sürdürülebilir kod oluşturur.

Prensip	Odak Noktası	Ana Kazanım	Uygulama Alanı
SRP	Tek Sorumluluk	Bakımı kolay kod	Sınıf tasarımı
OCP	Genişletmeye açık	Değişime dayanıklı yapı	Polimorfizm
LSP	Güvenli kalıtım	Tutarlı davranış	Inheritance
ISP	Odaklı arayüz	Az bağımlılık	Interface tasarımı
DIP	Soyutlama temelli	Esnek mimari	Dependency Injection

"

SOLID prensipleri, kodun bugünkü değil, yarınki halini korur.

— Temiz Kod Felsefesi

İyi kod, geliştirici değişse bile yaşamaya devam eden koddur.

SOLID prensipleri, yazılımımızı değişime uyumlu, genişletilebilir ve sürdürülebilir hale getiren temel yapı taşlarıdır.

Bu prensipler sayesinde kodumuz zaman içinde daha güçlü hale gelir.

Modular Architecture: Katmanlı Yapı



Core Layer

Temel servisler, ağ işlemleri, veri modelleri ve tüm modüller tarafından paylaşılan ortak bileşenler bulunur.



Feature Modules

Login, Payment, Profile gibi bağımsız özellik modülleri. Her biri kendi sorumluluğunda çalışır.



Shared UI

Ortak tasarım bileşenleri, renkler, fontlar ve stil rehberi tüm uygulamada tutarlılık sağlar.

”

Kod yazmak bir defteri doldurmak deęil, geleceęe mektup yazmaktır. Bugün yazdığınız her satır, yarının mirasıdır.

— Sürdürülebilir Geliştirme Felsefesi

Bu eğitimde öğrendiğimiz Clean Code, SOLID prensipleri, Cohesion/Coupling dengesi, Katmanlı Mimari ve Gitflow ile artık sadece çalışan kod deęil, yaşayan ve büyüyen kod yazabilirsiniz. Yazılım geliştirme, sadece teknik bir süreç deęil, geleceęi şekillendiren bir sanattır.